Язык программирования Python

Работа с файлами

Доцент кафедры ИВТ, к.т.н.
Проскурин Александр Викторович

Файлы

Чаще всего данные для обработки поступают из внешних источников – файлов.

Существуют два основных типа файлов: текстовые и бинарные.

- **Текстовые файлы** хранят только строковую информацию (тип str в языке Python) и открываются в любом текстовом редакторе (Блокноте, Notepad++ и т.д.). У текстовых файлов могут быть различные расширения: .txt, .csv, .html и многие другие (.py также является текстовым файлом).
- **Бинарные файлы** хранят информацию так же, как она хранится в оперативной памяти компьютера (нет необходимости в преобразовании к строковому типу данных), но нужно знать размер каждого сохраненного данного и порядок их записи в файле. В связи с этим для открытия бинарных файлов нужны специальные программы (например, MS Word для открытия файлов .doc, музыкальный проигрыватель для открытия .mp3 файлов и т.д.).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Работа с файлами

В Python при работе с файлами необходимо соблюдать определённую последовательность действий:

1. **Открыть файл.** Для этого используется функция open(). Если файл открывается для записи данных, то он будет создан автоматически. Если для чтения, то файл должен быть создан заранее, иначе произойдёт ошибка.
2. **Считать данные** из файла или **записать данные** в файл. При работе с текстовыми файлами чтение происходит с помощью методов read() / readlines() / readline(), а запись через методы write() / writelines(). Чтение и запись ведутся через специальный буфер с помощью функций операционной системы.
3. **Закрыть файл.** Открытый файл всегда необходимо закрывать, поскольку при открытии происходит блокировка работы с файлом других программ. Для явного закрытия используется метод close().

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Открытие файла. open()

Для открытия файла используется функция **open()**, которая возвращает **файловый объект**.

Этот объект связан с открытым файлом и хранит в себе **курсор** – подобие закладки в книге, указывающее на место, в котором было закончено чтение / запись.

Строка кода с использованием функции open() выглядит так:

```
переменная = open(имя_файла, режим_доступа, encoding=кодировка)
```

В коде **переменная** – это файловый объект, с помощью которого происходит вся дальнейшая работа с файлом.

имя файла является обязательным и указывается всегда.

режим доступа и **кодировка** не обязательны и могут быть пропущены.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Открытие файла. Аргументы

Имя файла может быть:

- **Относительным** (например, 'data.txt'). В этом случае путь к файлу указывается относительно папки, в которой находится программа.
- **Абсолютным** (начиная с корневого каталога: 'C:/Work/data.txt').

Исходя из того, с какой целью вы открываете файл, необходимо указать тот или иной **режим доступа**. Для указания режима доступа возможно использование нескольких символов и их комбинаций (по одной из каждого столбца):

'r' – открыть файл для чтения (используется по умолчанию)	't' – текстовый режим (используется по умолчанию)	'+' – открывает файл для обновления (чтение и запись)
'w' – открыть файл для записи	'b' – бинарный режим	
'a' – открыть файл для добавления в конец		

При чтении файла можно столкнуться с тем, что его **кодировка** не совпадает с той, что указана по умолчанию. Явным образом её можно задать через параметр encoding функции open(). Например:

```
encoding='cp1251'  или  encoding='utf8'
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Чтение данных

Для чтения данных файл должен быть открыт с режимом «rt» (read text), после чего можно прочитать содержимое одним из четырёх способов:

1. read() – считывает **всё содержимое** файла в одну строку (str).

2. read(количество_символов) – считывает в одну строку заданное **количество символов** из текстового файла. При повторном вызове будут считываться следующие символы.

3. readlines() – построчно считывает **весь файл в список строк**. Каждый элемент списка – одна строка исходного файла.

4. readline() – считывает **одну (текущую) строку** из файла. После этого внутренний курсор (закладка) смещается. При следующем вызове функции будет считана новая строка.

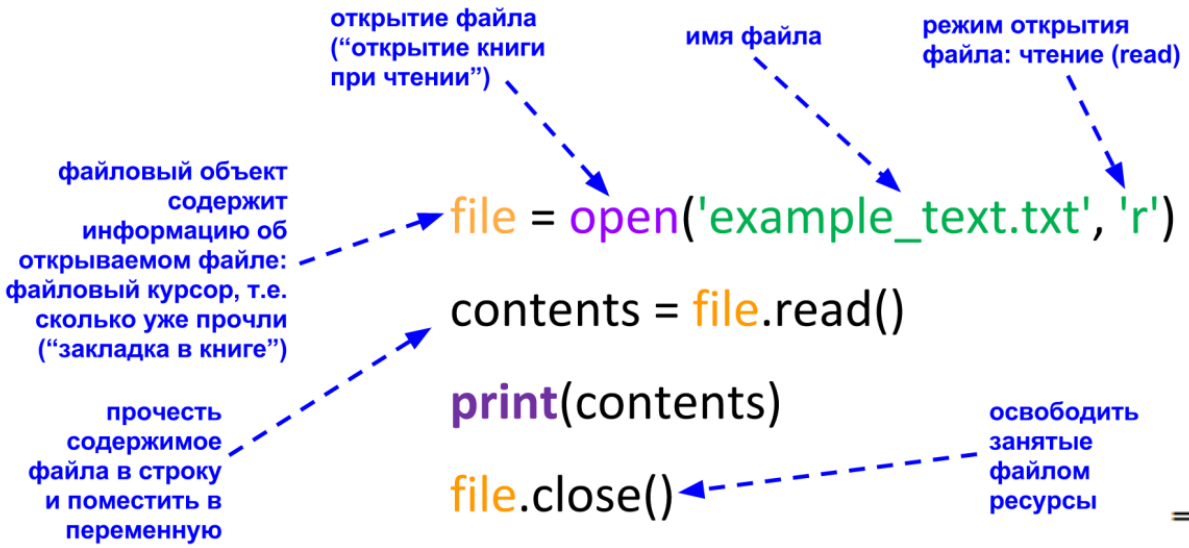
© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Чтение данных. read()

Создадим в папке файл с программой и текстовый файл «example_text.txt», внутри которого пропишем 3 строки:

```
Первая строка текста
Вторая строка текста
Третья строка текста
```

Программа состоит из следующего кода:



Результат работы:

```
=====================
Первая строка текста
Вторая строка текста
Третья строка текста
>>>
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Менеджер контекста

Обратите внимание, что необходимо обязательно закрывать файл вызовом метода close().

Более предпочтительным является использование менеджера контекста (ключевое слово **with**).

Преимущество этого способа в том, что файл всегда корректно закрывается после выполнения блока, либо если при выполнении было порождено исключение (ошибка).

В этом случае можно переписать код следующим образом:

```
contents = ''
with open('example_text.txt', 'r') as file:
    contents = file.read()
print(contents)
```

Файловый объект

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Чтение данных. read(num)

Метод read() может считывать не только весь файл, но и определённое количество символов. Рассмотрим на примере того же текстового файла «example_text.txt».

При открытии текстового файла для чтения, режим доступа можно опустить.

```
first_six = ''
rest = ''
with open('example_text.txt') as file:
    first_six = file.read(6)
    rest = file.read()
print('Первые 6 символов:', first_six)
print('Остальные символы:', rest)
```

Результат:

```
Первые 6 символов: Первая
Остальные символы:  строка текста
Вторая строка текста
Третья строка текста
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Чтение данных. readlines()

Если каждую строку файла нужно обрабатывать отдельно, то удобнее считать информацию в список строк. Для этого используется метод readlines():

```
lines = []
with open('example_text.txt', 'rt') as file:
    lines = file.readlines()
print(lines)
lines = list(map(str.strip, lines))
print(lines)
```

Удаляем пробельные символы в строках.

Результат работы кода:

```
['Первая строка текста\n', 'Вторая строка текста\n',
'Третья строка текста']
['Первая строка текста', 'Вторая строка текста',
'Третья строка текста']
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Чтение данных. readline()

Для чтения одной текущей строки используется метод readline(). При этом его можно не вызывать явным образом, а поместить файловый объект в цикл for:

```
lines = []
with open('example_text.txt', 'rt') as file:
    line = file.readline()
    print('Первая строка:', line)
    for line in file: # В line текущая строка
        lines.append(line)
print(lines)
```

Результат:

```
Первая строка: Первая строка текста

['Вторая строка текста\n', 'Третья строка текста']
```

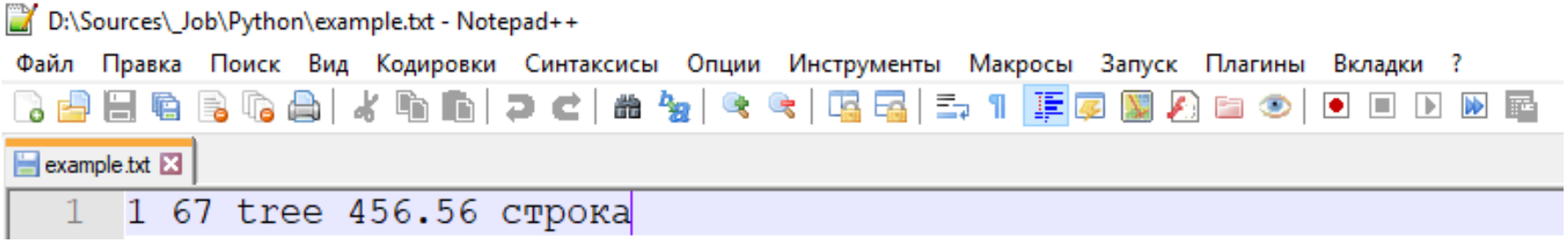

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Запись данных. write()

Для записи текстовых данных файл должен быть открыт с режимом «wt» (write text). После этого можно записать информацию методом write() или writelines().

Метод **write()** принимает только строковый тип данных, поэтому необходимо все записываемые элементы предварительно привести к типу str:

```
ls = [1, 67, 'tree', 456.56, 'строка']
st = [str(i) for i in ls] # Приведение к типу str
st = ' '.join(st) #Для примера объединим в одну строку
with open('example.txt', 'wt') as file:
    file.write(st)
```

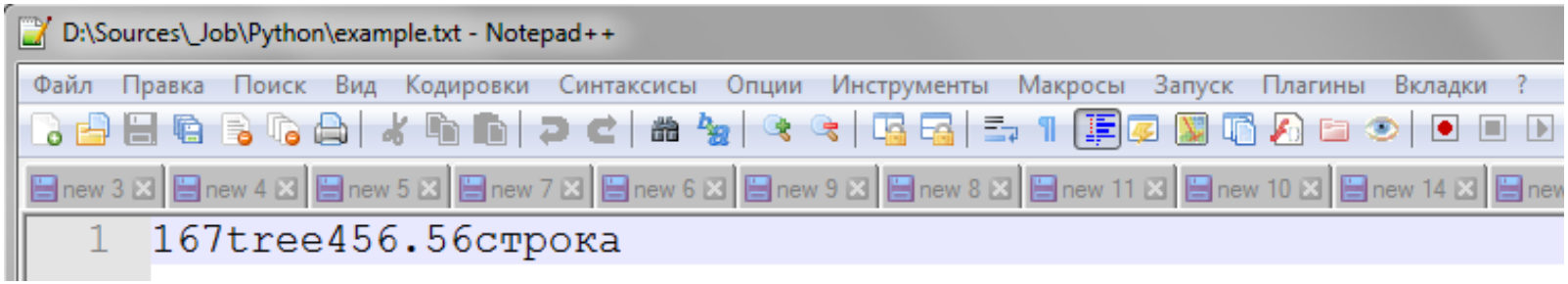


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Запись данных. writelines()

Метод **writelines()** принимает только список строк, поэтому как и при использовании write() необходимо все записываемые элементы предварительно привести к типу str:

```
ls = [1, 67, 'tree', 456.56, 'строка']
st = [str(i) for i in ls] # Приведение к типу str
with open('example.txt', 'wt') as file:
    file.writelines(st)
```



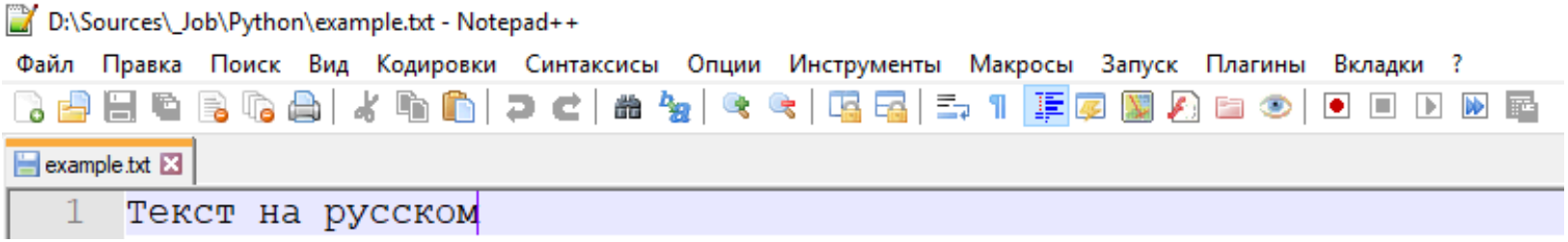
© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Кодировка

В некоторых случаях (например, если текстовый файл был создан в Linux, а открыть его пытаются в Windows) может возникнуть проблема с кодировкой (неправильная интерпретация символов). В этом случае можно явно **указать** необходимую **кодировку** при открытии файла через параметр **encoding**.

Пример. Создадим файл, записав в него информацию в кодировке utf8. После этого попробуем прочитать данные, но открыв файл в кодировке cp1251.

```
with open('example.txt', 'wt', encoding='utf8') as file:
    file.write('Текст на русском')
with open('example.txt', 'rt', encoding='cp1251') as file:
    print(file.read())
```



UTF8

cp1251

```
In [17]: runfile('D:/Sources/_Job/Python/t1.py', wdir='D:/Sources/_Job/Python')
РўРµРєСЃС, РЅР° СЂСѓСЃСЃРєРѕРј
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Бинарные файлы

Бинарные файлы, в отличие от текстовых, хранят информацию в виде набора байт. Для работы с ними в Python необходим встроенный модуль **pickle**. Этот модуль предоставляет две функции:

- `dump(obj, file)` – записывает объект obj в бинарный файл file.
- `load(file)` – считывает данные из бинарного файла в объект.

При открытии бинарного файла на чтение или запись нужно применять режим **«b»** в дополнение к режиму записи («w») или чтения («r»).

```
import pickle

FN = 'user.dat'
name = 'Роман'
age = 19

with open(FN, 'wb') as file:
    pickle.dump(name, file)
    pickle.dump(age, file)

with open(FN, 'rb') as file:
    name = pickle.load(file)
    age = pickle.load(file)
    print('Имя:', name,
          '\tВозраст:', age)
```

```
Имя: Роман      Возраст: 19
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример

Есть файл с текстом, в котором нужно найти все слова длиннее 2 символов. Дополнительно нужно удалить все символы пунктуации и лишние пробелы.

```
import string # Нужен для удаления знаков пунктуации и т.д.
strip = (string.whitespace + string.punctuation +
         string.digits + '"\'')
filename = 'D:/Documents/test.txt'
words = [] # Здесь накапливаем слова
with open(filename, encoding='utf8') as file:
   for line in file: # Перебираем файл по строкам
      for word in line.lower().split(): #Разбиваем на слова
         word = word.strip(strip) # Убираем лишние символы
         if len(word) > 2: # Если слово больше 2 символов
            words.append(word)
for word in sorted(words):
   print(word)
```

```
Ответ:
...
'ярости'
'ярость'
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева